

SugarPy Meetup — FastAPI + OpenAI Summarizer: From Prompt to Deployment

SugarPy Meetup

by: Arbin Bulaybulay, Senior Full Stack SE

Introduction

This document guides meetup attendees through building a FastAPI app that uses OpenAI's API to summarize text input.

Prompt (used to generate the app)

Build me a FastAPI app that uses OpenAI's API to summarize text input.

Requirements:

- Use FastAPI for the web framework
- Use Pydantic BaseModel for request validation
- Add an endpoint POST /summarize that accepts JSON {"text": "..."} and returns a summarized version
- Add GET /health endpoint that returns {"status": "ok", "message": "API is healthy!"}
- Add GET / that returns a welcome message
- Load the OpenAI API key from an environment variable (OPENAI_API_KEY), support .env for local dev
- Use uvicorn as the server

Also provide:

1. A requirements.txt with necessary dependencies
2. A render.yaml file so I can deploy this on Render free tier, with healthcheck set to /health
3. Clear instructions for testing with Insomnia/Postman (POST request with JSON body)

Generated file: main.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from dotenv import load_dotenv
import os
from openai import OpenAI

load_dotenv()
app = FastAPI()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

class TextInput(BaseModel):
    text: str

@app.get("/")
def read_root():
    return {"message": "Welcome to the FastAPI Summarizer API!"}

@app.get("/health")
def health_check():
    return {"status": "ok", "message": "API is healthy!"}

@app.post("/summarize")
```

```
def summarize(input: TextInput):
    try:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "system", "content": "Summarize the following text:"},
                      {"role": "user", "content": input.text}]
        )
        summary = response.choices[0].message.content
        return {"summary": summary}
    except Exception as e:
        return {"error": str(e)}
```

Generated file: requirements.txt

```
fastapi
uvicorn
openai
python-dotenv
```

Generated file: render.yaml

```
services:
- type: web
  name: fastapi-openai-summarizer
  env: python
  plan: free
  buildCommand: pip install -r requirements.txt
  startCommand: uvicorn main:app --host 0.0.0.0 --port 10000
  healthCheckPath: /health
  envVars:
    - key: OPENAI_API_KEY
      sync: false
```

Local setup & testing (before deployment)

1. Create a project folder and add main.py, requirements.txt, render.yaml.
2. Create a .env file (for local testing only) with:
OPENAI_API_KEY=your_openai_api_key_here
3. Install dependencies:
pip install -r requirements.txt
4. Run locally:
uvicorn main:app --reload --port 8000
5. Test endpoints:
GET http://127.0.0.1:8000/health
POST http://127.0.0.1:8000/summarize
Body (JSON): { "text": "Your long text here..." }

Deploy to Render (step-by-step)

1. Push your project to GitHub (see next section for git commands).
2. Log in to <https://render.com> and create a New Web Service.
3. Connect your GitHub repository and select the branch (main).
4. Render will detect render.yaml and configure the service.
5. In the Render dashboard, add an Environment Variable:

```
OPENAI_API_KEY = your_openai_api_key_here
6. Deploy; Render will build (pip install -r requirements.txt) and start the service.
7. After deployment, check the health endpoint:
  GET https://<your-render-app>.onrender.com/health
8. Test summarizer:
  POST https://<your-render-app>.onrender.com/summarize
  Body (JSON): { "text": "Sample text to summarize." }
```

Git & GitHub quick commands

```
# Initialize and push (if starting from local)
git init
git add .
git commit -m "Initial commit - FastAPI OpenAI Summarizer"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main

# If pushing to main is blocked by repo protection rules:
git checkout -b dev
git push -u origin dev
# Then open a Pull Request on GitHub and merge dev into main.
```

Testing with Insomnia / Postman / curl

```
Health check (GET):
curl -X GET https://<your-render-app>.onrender.com/health
```

```
Summarize (POST):
curl -X POST https://<your-render-app>.onrender.com/summarize \
  -H "Content-Type: application/json" \
  -d '{"text": "Long text to summarize"}'
```

In Insomnia/Postman: set method POST, URL `https://<your-render-app>.onrender.com/summarize`, Headers: Content-Type: application/json, Body: raw JSON `{ "text": "..."}.`

Prompt examples for attendees

- 1) News blurb:
'Artificial intelligence is transforming industries worldwide...'
- 2) Meeting notes:
'During today's team meeting, Sarah confirmed the marketing campaign launch date is October 15.'
- 3) Research abstract:
'This study investigates the impact of renewable energy integration into urban power grids...'

Notes & Best Practices

- Never commit your .env file or API keys to Git. Use .gitignore to exclude sensitive files.
- Use Render environment variables for production secrets.
- Monitor logs on Render to debug runtime errors (missing env vars, dependency issues).
- For heavier workloads or lower latency requirements, consider Cloud Run or a paid Render plan.